

Java Data Types

Instructor: Dr. Sayyad Najafi
Teaching Assistant: Ramin Iranmanesh

Data Types

Primitive Types:

These types store simple values directly in memory and have high performance. These types are divided into numeric and non-numeric categories.

Reference Types:

These types store the address (reference) of an object in memory. Their default value is null.

Primitive Integer Types

byte

- Size: 8 bits
- Range: -128 to 127

short

- Size: 16 bits
- Range: -32,768 to 32,767

int

- Size: 32 bits
- Range: -2,147,483,648 to 2,147,483,647

long

- Size: 64 bits
- Range: -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807

Primitive Floating-Point Types

float

- Size: 32 bits
- Range: approx. $1.4e-45$ to $3.4e+38$
- Description: Single-precision floating-point

double

- Size: 64 bits
- Range: approx. $4.9e-324$ to $1.8e+308$
- Description: Double-precision floating-point

Primitive Non-Numerical Types

char

- Size: 16 bits
- Range: '\u0000' to '\uffff' (0 to 65,535)
- Description: A single Unicode character

boolean

- Size: JVM-dependent (typically 1 byte or 1 bit)
- Values: true or false
- Description: Logical value (truth value)

Primitive Types Summary Table

Type	Size (bits)	Range / Values	Default Value
byte	8	-128 to 127	0
short	16	-32,768 to 32,767	0
int	32	-2,147,483,648 to 2,147,483,647	0
long	64	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	0L
float	32	approx. 1.4e-45 to 3.4e+38	0.0f
double	64	approx. 4.9e-324 to 1.8e+308	0.0d
char	16	'\u0000' to '\uffff' (0 to 65,535)	'\u0000'
boolean	JVM-dependent	true or false	false

Reference Types Categories

- **Class Types** – instances of classes
- **Interface Types** – references to objects that implement an interface
- **Array Types** – even arrays of primitives are objects
- **Enum Types** – a fixed set of named constants
- **Record Types** (Java 14+) – immutable data carriers with minimal boilerplate
- **Annotation Types** – used for metadata

Wrapper Classes

Each primitive type has a corresponding wrapper class:

- `byte` → `Byte`
- `short` → `Short`,
- `int` → `Integer`
- `long` → `Long`
- `float` → `Float`
- `double` → `Double`
- `char` → `Character`
- `boolean` → `Boolean`

Why use them?

- Required in generics
- Allow null as a value
- Provide utility methods